

Introduzione ad Arduino Programmazione e Componenti di Base

Prof. Bernardis Pierluigi

Indice

1	Introduzione ad Arduino	5
1.1	La scheda Arduino Uno	5
1.2	Alimentazione della scheda	5
1.3	I pin di alimentazione	7
1.4	I pin digitali	8
1.5	I pin analogici	10
1.6	Pin di comunicazione seriale (TX e RX)	12
1.7	Pin di interrupt	15
1.8	L'ambiente di sviluppo Arduino IDE	15
1.9	Struttura fondamentale di un programma	15
2	LED	17
2.1	Introduzione	17
2.2	Struttura fisica e principio di funzionamento	17
2.3	Parametri elettrici fondamentali	19
2.4	La resistenza di limitazione	19
2.5	Schema di collegamento	21
2.6	LED RGB	22
2.7	Programmazione con Arduino	23
2.7.1	Lampeggio del LED e funzione <code>delay/(ms);</code>	24
2.7.2	Variare la luminosità: PWM e <code>analogWrite()</code>	26
2.7.3	LED RGB	27
3	Pulsante	29
3.1	Introduzione	29
3.2	Struttura e funzionamento	29
3.3	Arduino e resistenze di pull-up e pull-down	30
3.3.1	Pull-Up interno ad arduino	31
3.4	Programmazione con Arduino	32
3.5	Il problema del rimbalzo - Bouncing	32

Capitolo 1

Introduzione ad Arduino

1.1 La scheda Arduino Uno

Panoramica dei componenti principali

Il microcontrollore ATmega328P

Versioni e differenze principali

1.2 Alimentazione della scheda

Per funzionare correttamente, la scheda Arduino Uno necessita di un'alimentazione a corrente continua con una tensione compresa tra 5 V e 12 V (a seconda della modalità scelta). La scheda è progettata per essere alimentata in tre modi diversi, ciascuno con caratteristiche e limiti specifici.

Modalità di alimentazione: USB, Vin e alimentazione esterna

Arduino Uno può ricevere energia elettrica attraverso tre differenti ingressi:

1. **Porta USB (tipo B):** quando la scheda è collegata al computer tramite cavo USB, riceve una tensione stabile di 5 V direttamente dalla porta USB del computer. Questa è la modalità più semplice e sicura per alimentare la scheda durante la fase di sviluppo e caricamento degli sketch. La corrente disponibile è limitata a 500 mA dalle specifiche USB 2.0.
2. **Connettore di alimentazione esterna (jack-barrel DC):** è il connettore cilindrico di colore nero presente sulla scheda (diametro esterno 5.5 mm, diametro interno 2.1 mm, positivo al centro). Accetta tensioni continue comprese tra 7 V e 12 V. La tensione in ingresso viene poi ridotta a 5 V dal regolatore di tensione a bordo.
3. **Pin Vin:** è un pin di ingresso presente sul connettore POWER della scheda. Può essere utilizzato per fornire una tensione non regolata (sempre tra 7 V e 12 V) direttamente da una batteria o da un alimentatore esterno, bypassando il jack DC ma passando comunque attraverso il regolatore di tensione.

Modalità	Tensione in ingresso	Note
Porta USB	5 V (stabile)	Il computer limita la corrente
Jack DC (consigliato)	da 7 V a 12 V	Tensione > 12 V surriscalda il regolatore
Pin Vin	da 7 V a 12 V	Non protegge da inversione di polarità

Attenzione – Pin 5V come ingresso

Il pin etichettato **5V** sul connettore **POWER** è un'**uscita** di tensione regolata a 5 V, non un ingresso. Fornire tensione direttamente su questo pin è **sconsigliato** perché si bypassano le protezioni del regolatore e si rischia di danneggiare irreversibilmente il microcontrollore se la tensione non è perfettamente stabile a 5 V.

Il regolatore di tensione a bordo

Quando la scheda viene alimentata tramite jack DC o pin Vin con una tensione superiore a 5 V, entra in funzione il **regolatore di tensione lineare** integrato sulla scheda. Su Arduino Uno si tratta tipicamente di un componente siglato **NCP1117ST50T3G** o **AMS1117-5.0**, in grado di fornire una tensione stabile di 5 V a partire da un ingresso compreso tra 7 V e 12 V.

Il principio di funzionamento di un regolatore lineare è semplice: la tensione in eccesso ($V_{in} - 5\text{ V}$) viene *dissipata* sotto forma di calore. Maggiore è la differenza tra tensione in ingresso e tensione di uscita, maggiore sarà il calore prodotto.

$$P_{\text{dissipata}} = (V_{in} - 5\text{ V}) \times I_{\text{assorbita}} \quad (1.1)$$

Ad esempio, se alimentiamo la scheda a 12 V e il circuito assorbe complessivamente 200 mA, il regolatore deve dissipare:

$$P = (12\text{ V} - 5\text{ V}) \times 0.2\text{ A} = 1.4\text{ W}$$

Questo livello di potenza dissipata è già significativo per un piccolo componente senza dissipatore, e può portare a temperature elevate che riducono la vita utile del regolatore e della scheda stessa. Per questo motivo è fortemente consigliato non superare i 12 V di tensione in ingresso quando si alimenta Arduino tramite jack DC o pin Vin.

Perché non superare i 12 V?

Il regolatore di tensione di Arduino Uno può teoricamente accettare tensioni fino a 20 V (limite massimo assoluto del datasheet), ma nella pratica tensioni superiori a 12 V causano un surriscaldamento eccessivo del componente, con conseguente riduzione della vita utile della scheda o spegnimento per protezione termica. Per alimentazioni superiori a 12 V è preferibile utilizzare un regolatore switching esterno.

Selezione automatica della sorgente di alimentazione

Arduino Uno è dotato di un circuito di **selezione automatica della sorgente di alimentazione**, realizzato tramite un amplificatore operazionale (comparatore) e un MOSFET a canale P. Questo circuito decide quale delle due possibili fonti di energia utilizzare:

- Se è presente tensione sul jack DC (o sul pin Vin) superiore a circa 6.6 V, il circuito scollega automaticamente l'alimentazione proveniente dalla porta USB e utilizza la tensione esterna.
- Se la tensione esterna scende al di sotto di tale soglia (o è assente), il circuito commuta sull'alimentazione USB.

Questa commutazione avviene in modo completamente trasparente per l'utente e garantisce che la scheda possa essere alimentata in modo sicuro anche quando sono collegate entrambe le sorgenti (ad esempio durante il caricamento di uno sketch da computer mentre la scheda è già alimentata da una batteria esterna).

1.3 I pin di alimentazione

Sul connettore **POWER** di Arduino Uno sono disponibili diversi pin dedicati all'alimentazione del circuito. Questi pin possono essere utilizzati per prelevare tensione stabilizzata da destinare ai componenti esterni (sensori, LED, moduli) oppure per fornire alimentazione alla scheda stessa.

Pin 5V e 3.3V: caratteristiche e limiti di corrente

I pin contrassegnati con 5V e 3.3V forniscono tensioni stabilizzate e possono essere utilizzati per alimentare circuiti esterni compatibili con tali livelli di tensione.

- **Pin 5V:** fornisce una tensione stabilizzata di 5 V. Questa tensione è generata dal regolatore lineare integrato sulla scheda (quando l'alimentazione proviene da jack DC o Vin) oppure proviene direttamente dalla porta USB (quando la scheda è alimentata via USB).
- **Pin 3.3V:** fornisce una tensione stabilizzata di 3.3 V, generata da un secondo regolatore lineare (tipicamente un LP2985) che preleva l'ingresso dalla linea a 5 V.

Pin	Tensione	Corrente massima consigliata
5V	5 V	500 mA (USB) / 800 mA (esterna)
3.3V	3.3 V	50 mA

Attenzione – Non fornire tensione sui pin 5V o 3.3V

I pin 5V e 3.3V sono **uscite** di tensione regolata. Non devono essere utilizzati come ingressi per alimentare la scheda. Fornire una tensione esterna su questi pin, specialmente se superiore al valore nominale, può danneggiare irreversibilmente il regolatore e il microcontrollore.

Pin Vin: ingresso per alimentazione non regolata

Il pin **Vin** (abbreviazione di *Voltage input*) è un ingresso di alimentazione non regolata. Può essere utilizzato per fornire tensione alla scheda quando non si impiega il jack DC o la porta USB. Le caratteristiche principali sono:

- **Tensione di ingresso consigliata:** da 7 V a 12 V. Tensioni inferiori a 7 V potrebbero non garantire il corretto funzionamento del regolatore a 5 V, causando instabilità o reset del microcontrollore.
- **Tensione massima assoluta:** 20 V. Tuttavia, come discusso in precedenza, tensioni superiori a 12 V provocano un'eccessiva dissipazione di potenza nel regolatore, con conseguente surriscaldamento.
- **Protezione da inversione di polarità:** il pin **Vin** **non** è protetto da un diodo contro l'inversione di polarità (a differenza del jack DC). Collegare accidentalmente il positivo della batteria a GND e il negativo a Vin può distruggere istantaneamente il regolatore e altri componenti.

Pin GND: massa comune del circuito

I pin etichettati **GND** (dall'inglese *ground*, massa) costituiscono il riferimento comune a 0 V per l'intero circuito. Su Arduino Uno sono presenti tre pin **GND** sul connettore **POWER** e altri due pin **GND** sui connettori digitali (vicino al pin 13 e al pin **AREF**). Tutti i pin **GND** sono collegati internamente alla stessa massa.

Buona pratica – Masse comuni

Quando si utilizza un'alimentazione esterna per pilotare carichi di potenza (motori, relè, strisce LED), è **obbligatorio** collegare la massa dell'alimentazione esterna a un pin **GND** di Arduino. Questo garantisce che i segnali di controllo inviati da Arduino abbiano lo stesso riferimento di tensione del circuito di potenza, evitando malfunzionamenti o letture errate.

Corrente massima erogabile e protezioni

La scheda Arduino Uno incorpora alcune protezioni per limitare i danni in caso di utilizzo improprio, ma non è progettata per resistere a condizioni di sovraccarico prolungato.

- **Fusibile ripristinabile:** sulla linea di alimentazione a 5 V proveniente dalla porta USB è presente un fusibile a coefficiente di temperatura positivo (PTC). Questo componente aumenta la propria resistenza quando la corrente supera una soglia (tipicamente 500 mA), limitando la corrente e proteggendo la porta USB del computer. Quando il sovraccarico cessa, il componente si raffredda e ripristina la conduzione normale. Il fusibile **non** protegge il regolatore a 5 V quando la scheda è alimentata tramite jack DC o Vin.
- **Diodo di protezione sul jack DC:** un diodo in serie al jack DC impedisce che una tensione inversa applicata accidentalmente raggiunga il regolatore. Tuttavia, questo diodo introduce una caduta di tensione di circa 0.7 V, che deve essere considerata nella scelta dell'alimentatore.
- **Diodi di clamping sui pin di I/O:** ciascun pin digitale è protetto da diodi interni che limitano la tensione a un massimo di $V_{cc} + 0.5 \text{ V}$ e a un minimo di -0.5 V . Questi diodi proteggono il microcontrollore da scariche elettrostatiche (ESD) e da brevi sovratensioni, ma non sono in grado di dissipare potenze significative. Applicare tensioni negative o superiori a 5.5 V per periodi prolungati può distruggere i diodi e il pin corrispondente.

Riepilogo – Limiti operativi assoluti

Parametro	Limite
V_{MAX} su jack DC / Vin	20 V (assoluto), 12 V raccomandato
V_{MIN} su jack DC / Vin	7 V
I_{MAX} da pin 5V	800 mA (dipende da Vin e temperatura)
I_{MAX} da pin 3.3V	50 mA
I_{MAX} totale da tutti i pin di I/O	200 mA

1.4 I pin digitali

I pin digitali di Arduino Uno sono i connettori contrassegnati dai numeri da 0 a 13 sul bordo superiore e inferiore della scheda (Figura ref pinout). Ciascuno di questi pin può essere configurato via software per comportarsi come **ingresso** (INPUT) o come **uscita** (OUTPUT). Quando configurato come uscita, il pin può fornire una tensione di 5 V (livello logico HIGH) o 0 V (livello logico LOW). Quando configurato come ingresso, il pin è in grado di leggere la tensione presente sul terminale e interpretarla come un segnale logico HIGH o LOW.

Configurazione come ingresso o uscita

La direzione del flusso del segnale è definita via software, ma corrisponde a una precisa configurazione hardware interna al microcontrollore. Ogni pin digitale è collegato a due sottocircuiti principali: uno di uscita (driver) e uno di ingresso (buffer di lettura).

1. **Configurazione come uscita (OUTPUT):** viene attivato il circuito di uscita, costituito da una coppia di transistor che collegano il pin alla tensione di alimentazione oppure a massa. Il microcontrollore impone quindi direttamente il livello logico del pin (HIGH $\approx 5\text{ V}$, LOW $\approx 0\text{ V}$) ed è in grado di fornire o assorbire corrente verso il circuito esterno.
2. **Configurazione come ingresso (INPUT):** il circuito di uscita viene disattivato, quindi il microcontrollore non impone alcun livello di tensione sul pin. Viene invece attivato il circuito di ingresso, che misura la tensione presente sul pin senza influenzarla. L'impedenza vista dall'esterno risulta molto elevata e la corrente assorbita è trascurabile.

Quando un pin è configurato come ingresso, il suo livello logico dipende esclusivamente dal circuito esterno. Se il pin non è collegato a una sorgente definita, la tensione può assumere valori imprevedibili a causa dei disturbi elettrici.

Livelli di tensione e soglie logiche

Quando un pin digitale è configurato come **uscita**, i livelli di tensione sono ben definiti: 0 V per LOW e 5 V per HIGH (in realtà circa 4.8 V–4.9 V a causa della caduta interna sul transistor di uscita).

Quando il pin è configurato come **ingresso**, la tensione applicata dall'esterno viene interpretata secondo le **soglie logiche** definite dal costruttore del microcontrollore ATmega328P:

Livello logico	Tensione in ingresso (V_{in})	Interpretazione
LOW	$V_{in} < 1.5\text{ V}$	Il pin legge 0 logico
HIGH	$V_{in} > 3.0\text{ V}$	Il pin legge 1 logico

Attenzione – La zona proibita

Qualsiasi tensione compresa tra 1.5 V e 3.0 V cade in una **zona indeterminata**: il microcontrollore potrebbe interpretare il segnale in modo casuale come HIGH o LOW, generando comportamenti imprevedibili. Per questo motivo è necessario assicurarsi che i segnali digitali in ingresso abbiano transizioni nette tra i due stati logici.

Corrente massima per pin e per l'intera scheda

I pin di Arduino non sono in grado di erogare o assorbire una quantità illimitata di corrente. Dal datasheet dell'ATmega328P e dalle specifiche della scheda Arduino Uno si ricavano i seguenti limiti operativi:

Parametro	Valore massimo
Corrente erogata da un singolo pin configurato come uscita HIGH	40 mA
Corrente assorbita da un singolo pin configurato come uscita LOW	40 mA
Corrente totale erogata dall'insieme di tutti i pin di uscita	200 mA

Attenzione – Superamento dei limiti di corrente

Superare anche per breve tempo i 40 mA su un singolo pin o i 200 mA complessivi può danneggiare irreversibilmente il microcontrollore o il regolatore di tensione della scheda. Per pilotare carichi che richiedono correnti superiori (motori, relè, strisce LED ad alta luminosità) è **obbligatorio** utilizzare componenti esterni come transistor o circuiti integrati driver.

Resistenze di pull-up interne

Quando un pin digitale è configurato come **ingresso** e non è collegato ad alcun circuito esterno (né a 5 V né a massa), la sua tensione non è definita e può variare in modo imprevedibile a causa dei campi elettromagnetici ambientali e dei disturbi elettrici. Questa condizione è nota come **pin flottante** e porta a letture casuali e inaffidabili.

Per evitare questo problema, il microcontrollore dispone di **resistenze di pull-up interne** attivabili via software. Una resistenza di pull-up è un resistore di elevato valore (tipicamente da 20 kΩ a 50 kΩ) che collega internamente il pin alla tensione di alimentazione 5 V.

Quando la resistenza di pull-up è attiva, in assenza di un circuito esterno che imponga un diverso livello di tensione, il pin assume un livello logico HIGH. Se invece il pin viene collegato a massa attraverso un circuito esterno a bassa impedenza, la tensione scende a 0 V (livello LOW), poiché la corrente che attraversa la resistenza di pull-up è molto piccola e non è sufficiente a mantenere il livello alto.

In questo modo si garantisce che il pin presenti sempre un livello logico definito, evitando condizioni di indeterminazione.

Buona pratica – Evitare pin flottanti

Ogni pin di ingresso **deve sempre** essere portato a un livello di tensione definito, o tramite un collegamento esterno (ad esempio con una resistenza di pull-down o pull-up) o attivando le resistenze di pull-up interne del microcontrollore. Non lasciare mai pin di ingresso scollegati.

1.5 I pin analogici

I pin analogici di Arduino Uno sono sei, contrassegnati dalle etichette **A0 – A5** sul lato sinistro della scheda. A differenza dei pin digitali, che possono assumere solo due stati logici (HIGH o LOW), i pin analogici sono in grado di misurare una **gamma continua di tensioni** compresa tra 0 V e un valore massimo di riferimento (tipicamente 5 V). Questa capacità è resa possibile dal **convertitore analogico-digitale (ADC)** integrato nel microcontrollore ATmega328P.

Il convertitore analogico-digitale (ADC) a 10 bit

Un convertitore analogico-digitale è un circuito elettronico che trasforma una tensione continua (segnale analogico) in un numero digitale (segnale discreto) che può essere elaborato dal microcontrollore. L'ADC dell'ATmega328P ha una risoluzione di **10 bit**. Ciò significa che il valore letto viene rappresentato con un numero intero a 10 bit, che può assumere $2^{10} = 1024$ valori distinti, da **0** a **1023**.

La relazione tra la tensione in ingresso V_{in} e il valore numerico restituito dall'ADC è lineare:

$$\text{valore ADC} = \frac{V_{in}}{V_{ref}} \times 1023 \quad (1.2)$$

dove V_{ref} è la **tensione di riferimento** dell'ADC. Conoscendo il valore letto e la tensione di riferimento, è possibile risalire alla tensione originale:

$$V_{in} = \frac{\text{valore ADC}}{1023} \times V_{ref} \quad (1.3)$$

Nota – Il valore 1023

Poiché i valori vanno da 0 a 1023, il massimo valore numerico corrisponde a $V_{in} = V_{ref}$. Alcune formule approssimano dividendo per 1024 invece di 1023, ma la differenza è trascurabile nella maggior parte delle applicazioni pratiche. La documentazione ufficiale di Arduino utilizza il fattore 1023.

Tensione di riferimento e risoluzione

La **tensione di riferimento** (V_{ref}) determina il fondo scala dell'ADC, ovvero la massima tensione che può essere misurata. Per impostazione predefinita, Arduino Uno utilizza la propria tensione di alimentazione come riferimento, ovvero 5 V. In questa configurazione, l'ADC può misurare tensioni comprese tra 0 V e 5 V.

Con $V_{ref} = 5$ V, la **risoluzione** dell'ADC — ovvero la minima variazione di tensione che il convertitore è in grado di distinguere — è data da:

$$\text{Risoluzione} = \frac{V_{ref}}{1024} \approx \frac{5 \text{ V}}{1024} \approx 4.9 \text{ mV} \quad (1.4)$$

Ciò significa che variazioni di tensione inferiori a circa 5 mV non vengono rilevate, e il valore letto rimane invariato.

È possibile modificare la tensione di riferimento utilizzando il pin **AREF** (Analog Reference) o selezionando riferimenti interni alternativi tramite software.

Attenzione – Utilizzo di AREF

Se si applica una tensione di riferimento esterna sul pin **AREF**, è **obbligatorio** configurare il software per utilizzare il riferimento esterno **prima** di effettuare qualsiasi lettura analogica. In caso contrario, si rischia di danneggiare il microcontrollore a causa di un conflitto tra il riferimento interno e quello esterno.

Tempo di acquisizione e frequenza di campionamento

L'ADC dell'ATmega328P richiede un certo tempo per stabilizzare la lettura dopo una variazione del segnale di ingresso. La frequenza massima di campionamento è di circa 15 kHz (circa 15 000 letture al secondo) nella configurazione predefinita. Letture più rapide possono essere ottenute modificando i registri interni, ma a scapito della precisione.

Utilizzo come pin digitali aggiuntivi

I pin analogici A0 – A5 possono essere utilizzati anche come **pin digitali di I/O** aggiuntivi. In questa modalità, si comportano esattamente come i pin digitali standard (0–13), supportando le funzioni `pinMode()`, `digitalWrite()` e `digitalRead()`. Per riferirsi a un pin analogico in modalità digitale, si può utilizzare l'etichetta corrispondente (es. A0) oppure il numero equivalente (per Arduino Uno, A0 corrisponde al pin 14, A1 al 15, e così via fino a A5 che corrisponde al pin 19).

Questa caratteristica è particolarmente utile quando i pin digitali standard sono già tutti occupati e si necessita di qualche I/O digitale supplementare.

Nota – PWM sui pin analogici

A differenza di alcuni pin digitali (3, 5, 6, 9, 10, 11), i pin analogici **non** supportano l'uscita PWM (`analogWrite()`). Tentare di utilizzare `analogWrite()` su un pin analogico non produrrà l'effetto desiderato.

Pin analogico	Equivalente digitale	Supporta PWM?
A0	14	No
A1	15	No
A2	16	No
A3	17	No
A4	18	No (utilizzato anche per I ² C SDA)
A5	19	No (utilizzato anche per I ² C SCL)

Attenzione – Pin A4 e A5 e comunicazione I²C

I pin analogici **A4** (SDA) e **A5** (SCL) sono condivisi con l'interfaccia di comunicazione I²C (TWI). Se si utilizzano dispositivi I²C (come display LCD o sensori), questi pin non sono disponibili per l'uso come I/O digitale generico. In caso di conflitto, è possibile utilizzare pin analogici differenti o disabilitare l'I²C via software.

1.6 Pin di comunicazione seriale (TX e RX)

La comunicazione seriale è uno dei metodi più semplici e diffusi per trasmettere dati tra due dispositivi. Arduino Uno dispone di una porta seriale hardware integrata nel microcontrollore, accessibile attraverso i pin digitali 0 e 1, contrassegnati rispettivamente con le etichette **RX** (Receive) e **TX** (Transmit).

La porta seriale hardware USART

All'interno del microcontrollore ATmega328P è presente un modulo hardware dedicato denominato **USART** (*Universal Synchronous and Asynchronous serial Receiver and Transmitter*). Questo circuito è in grado di gestire la trasmissione e la ricezione di dati seriali in modo autonomo, senza impegnare la CPU per ogni singolo bit trasmesso.

Nella modalità più comune, detta **asincrona**, la comunicazione avviene secondo il seguente schema:

- I dati vengono trasmessi un bit alla volta su un singolo filo (più un filo di massa come riferimento).
- Il trasmettitore e il ricevitore devono concordare preventivamente una **velocità di trasmissione** (*baud rate*), espressa in bit al secondo (bps).
- Ogni byte di dati è incorniciato da un **bit di start** (sempre a livello logico LOW) e da uno o più **bit di stop** (sempre a livello logico HIGH).
- Opzionalmente può essere aggiunto un **bit di parità** per il rilevamento di errori di trasmissione.

Su Arduino Uno, la USART è tipicamente configurata con un baud rate di 9600 baud (9600 bit al secondo) e utilizza il formato **8-N-1**: 8 bit di dati, nessun bit di parità, 1 bit di stop.

Parametro	Valore tipico su Arduino Uno
Baud rate predefinito	9600 baud (configurabile fino a 115 200 baud)
Bit di dati	8
Bit di parità	Nessuno (N)
Bit di stop	1
Livelli di tensione	0 V (LOW) e 5 V (HIGH)

Approfondimento – Differenza tra USART e UART

Il termine **UART** (*Universal Asynchronous Receiver-Transmitter*) indica un modulo che gestisce esclusivamente la comunicazione asincrona. Il termine **USART** include anche la modalità sincrona, in cui viene trasmesso un segnale di clock separato. L'ATmega328P dispone di una USART, ma su Arduino viene utilizzata quasi esclusivamente in modalità asincrona (UART).

Pin 0 (RX) e pin 1 (TX)

I pin digitali **0 (RX)** e **1 (TX)** sono fisicamente collegati alla USART del microcontrollore e svolgono funzioni complementari:

- **Pin 0 – RX (Receive)**: è l'ingresso della porta seriale. Riceve i dati provenienti da un altro dispositivo (ad esempio un computer, un modulo GPS, un altro Arduino) e li rende disponibili al microcontrollore per la lettura.
- **Pin 1 – TX (Transmit)**: è l'uscita della porta seriale. Trasmette i dati generati da Arduino verso un dispositivo esterno.

Quando si utilizza la comunicazione seriale tra due dispositivi, il pin **TX** di un dispositivo deve essere collegato al pin **RX** dell'altro, e viceversa. I due dispositivi devono inoltre condividere un riferimento di massa comune (**GND**).

È importante notare che i pin 0 e 1 sono **condivisi** tra la comunicazione seriale con il computer (attraverso il chip di conversione USB-Seriale) e qualsiasi circuito esterno collegato a questi pin. Questa condivisione ha implicazioni pratiche rilevanti, discusse nella sottosezione seguente.

Interferenza durante il caricamento degli sketch

Quando si carica un nuovo programma su Arduino tramite la porta USB, i dati vengono inviati dal computer alla scheda attraverso la connessione seriale. Durante questa fase, il chip di conversione USB-Seriale (un ATmega16U2 su Arduino Uno Rev3) comunica direttamente con i pin **RX** e **TX** del microcontrollore ATmega328P.

Se in quel momento sono collegati componenti esterni ai pin 0 e 1, si possono verificare due tipi di interferenza:

1. **Disturbo del segnale di caricamento**: un circuito esterno collegato al pin **RX** (pin 0) potrebbe imporre un livello di tensione che interferisce con i dati in arrivo dal computer, causando errori o l'impossibilità di caricare lo sketch.
2. **Danneggiamento dei componenti esterni**: durante la programmazione, il pin **TX** (pin 1) trasmette dati verso il computer. Se a questo pin è collegato un componente sensibile (ad esempio un LED senza resistenza adeguata, o l'ingresso di un altro circuito a bassa tensione), la corrente di trasmissione potrebbe danneggiarlo.

Attenzione – Scollegare i circuiti esterni prima del caricamento

È buona norma **scollegare temporaneamente** qualsiasi circuito collegato ai pin 0 e 1 prima di caricare un nuovo sketch su Arduino. In alternativa, se il circuito esterno deve rimanere connesso, è possibile utilizzare pin seriali software (attraverso la libreria **SoftwareSerial**) su altri pin digitali, lasciando i pin 0 e 1 dedicati esclusivamente alla comunicazione con il computer.

Una volta completato il caricamento, la comunicazione seriale tra Arduino e il computer può proseguire normalmente per lo scambio di dati a runtime (ad esempio per visualizzare i valori letti da un sensore sul *Serial Monitor*).

La comunicazione con il computer tramite USB

Sebbene i pin 0 e 1 siano fisicamente i terminali della comunicazione seriale, l'utente interagisce con essi principalmente attraverso la **porta USB** della scheda. Questa apparente contraddizione è risolta dal **chip di conversione USB-Seriale** presente sulla scheda.

Su Arduino Uno Rev3, questo ruolo è svolto da un secondo microcontrollore, l'**ATmega16U2**, programmato per comportarsi come un ponte tra il protocollo USB (utilizzato dal computer) e il protocollo seriale TTL (utilizzato dall'ATmega328P). Il flusso dei dati è il seguente:

1. Il computer invia dati attraverso la porta USB emulando una porta seriale virtuale (COM su Windows, `/dev/tty*` su Linux e macOS).
2. Il chip ATmega16U2 riceve i dati via USB, li converte in segnali seriali TTL e li inoltra al pin RX (pin 0) dell'ATmega328P.
3. L'ATmega328P legge i dati dal pin RX e, se necessario, risponde trasmettendo dati sul pin TX (pin 1).
4. L'ATmega16U2 riceve i dati dal pin TX, li converte in pacchetti USB e li invia al computer.

Questo meccanismo è completamente trasparente per l'utente, che vede la scheda Arduino come una periferica seriale collegata al computer.

Nota – I LED TX e RX sulla scheda

Sulla scheda Arduino Uno sono presenti due piccoli LED contrassegnati con TX e RX. Questi LED lampeggiano ogni volta che viene trasmesso o ricevuto un dato attraverso la porta seriale USB. Il loro lampeggiamento è particolarmente utile per verificare visivamente che la comunicazione sia in corso, ad esempio durante il caricamento di uno sketch o durante lo scambio di dati con il *Serial Monitor*.

La comunicazione seriale via USB rappresenta lo strumento principale per il *debugging* dei programmi su Arduino: attraverso le funzioni `Serial.print()` e `Serial.println()` è possibile inviare messaggi e valori di variabili al computer, visualizzandoli nell'ambiente di sviluppo integrato.

1.7 Pin di interrupt

Concetto di interrupt hardware

Pin 2 e 3: interrupt esterni disponibili

Utilizzo tipico e vantaggi

1.8 L'ambiente di sviluppo Arduino IDE

Installazione e configurazione

Interfaccia principale: barra degli strumenti, editor e console

Selezione della scheda e della porta seriale

Verifica e caricamento di uno sketch

1.9 Struttura fondamentale di un programma

La funzione `setup()`: inizializzazione

La funzione `loop()`: esecuzione ciclica

Il ruolo delle librerie

Capitolo 2

LED

2.1 Introduzione

Il **LED** (*Light Emitting Diode*, ovvero Diodo ad Emissione di Luce) è uno dei componenti elettronici più utilizzati al mondo. Lo troviamo ovunque nella vita quotidiana: nei semafori stradali, nelle lampade a risparmio energetico, negli schermi degli smartphone, nei display dei televisori, nelle luci dei cruscotti delle automobili e persino nelle fibre ottiche che trasmettono dati su internet.

Il motivo del suo enorme successo è semplice: rispetto alle tradizionali lampadine a incandescenza, il LED consuma molto meno energia, dura molto più a lungo (fino a 50.000 ore di funzionamento) e si accende istantaneamente senza surriscaldarsi.

2.2 Struttura fisica e principio di funzionamento

Cos'è un diodo

Per capire un LED, dobbiamo prima capire cos'è un **diodo**. Un diodo è un componente elettronico che permette alla corrente elettrica di scorrere **in una sola direzione**, bloccandola nell'altra. Si comporta come una valvola a senso unico per la corrente.

Questa proprietà è dovuta alla struttura interna del diodo, realizzata con materiali **semiconduttori** (tipicamente il silicio o il germanio), ovvero materiali che si trovano a metà strada tra i conduttori (come il rame) e gli isolanti (come il vetro).

Perché un LED emette luce

Un LED è un tipo particolare di diodo che, quando la corrente lo attraversa nel verso corretto, emette luce. Senza entrare nei dettagli della fisica quantistica, possiamo capirne il funzionamento con un'analogia intuitiva.

Immaginiamo gli elettroni come palline che si muovono all'interno del materiale semiconduttore. Quando una pallina *cade* da un livello energetico alto a uno basso all'interno del materiale, rilascia energia. Nei normali resistori questa energia si trasforma in calore. In un LED, grazie alla scelta particolare dei materiali semiconduttori utilizzati (arseniuro di gallio, nitruro di gallio, ecc.), questa energia viene rilasciata sotto forma di **fotoni**, ovvero particelle di luce.

Il colore della luce emessa dipende direttamente dal materiale semiconduttore scelto: materiali diversi corrispondono a livelli energetici diversi, e quindi a colori diversi.

Colore	Materiale	Tensione tipica V_f
Rosso	Arseniuro di gallio-alluminio (AlGaAs)	da 1.8 V a 2.1 V
Arancione	Arseniuro di gallio-fosforo (GaAsP)	da 2.0 V a 2.2 V
Giallo	Fosforo di gallio (GaP)	da 2.1 V a 2.4 V
Verde	Nitrato di gallio-indio (InGaN)	da 2.2 V a 3.5 V
Blu	Nitrato di gallio (GaN)	da 3.0 V a 3.5 V
Bianco	LED blu + fosforo giallo	da 3.0 V a 3.5 V

Curiosità: il LED bianco

La luce bianca non esiste come singolo colore: è la sovrapposizione di tutti i colori dello spettro. I LED bianchi vengono costruiti combinando un LED blu ad alta efficienza con uno strato di materiale fosforescente giallo. La luce blu eccita il fosforo che emette luce gialla; la combinazione di blu e giallo produce la percezione del bianco da parte dell'occhio umano.

Polarità del LED: anodo e catodo

Come tutti i diodi, il LED è un componente **polarizzato**: ha un terminale positivo e uno negativo, e deve essere collegato nel verso giusto per funzionare.

- **Anodo (A)**: il terminale positivo. Si riconosce perché è la **gamba più lunga** del LED e all'interno del LED si nota che è presente una lamina chiamata **Lamina Minore**. Da questo terminale entra la corrente.
- **Catodo (C)**: il terminale negativo. Si riconosce perché è la **gamba più corta** del LED e all'interno del LED si nota che è presente una lamina chiamata **Lamina Maggiore**. Da questo terminale esce la corrente.

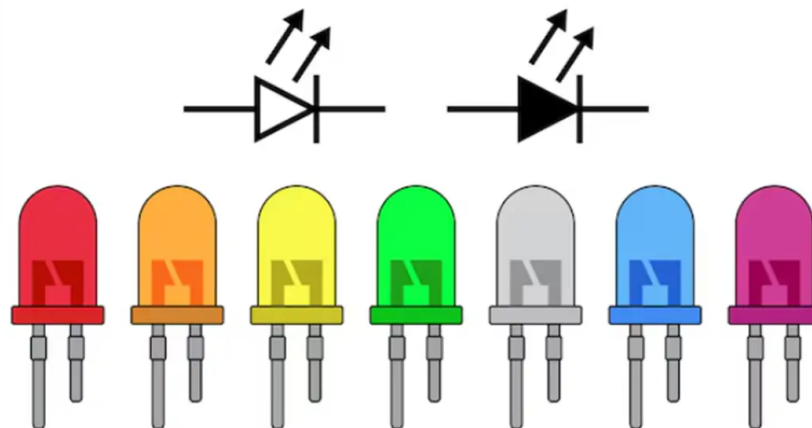


Figura 2.1: Struttura fisica di un LED standard da 5mm.

Attenzione - Polarità invertita

Se colleghiamo il LED con la polarità invertita (provando a far entrare la corrente dal catodo), il LED **non si accende** e blocca il passaggio della corrente. In questa condizione il LED non si danneggia, ma semplicemente non funziona. Tuttavia, se la tensione inversa applicata supera il valore massimo specificato nel datasheet (tipicamente 5 V per LED standard), il componente si danneggia irreversibilmente.

2.3 Parametri elettrici fondamentali

Prima di collegare un LED a qualsiasi circuito, è indispensabile conoscere i suoi parametri elettrici. Questi valori si trovano nel **datasheet** del componente, ovvero la scheda tecnica fornita dal produttore.

Tensione di polarizzazione diretta V_f

La **tensione di polarizzazione diretta** (*forward voltage*, V_f) è la caduta di tensione che si misura ai capi del LED quando è acceso e la corrente scorre nel verso corretto. Come abbiamo visto nella tabella precedente, questo valore dipende dal colore del LED e si aggira tipicamente tra 1.8 V e 3.5 V.

È fondamentale ricordare che V_f è una **proprietà fisica del componente**: non possiamo cambiarla. Qualunque sia la tensione di alimentazione del nostro circuito, la differenza di potenziale ai capi del LED *sarà sempre e solo* la sua V_f .

Corrente diretta nominale I_f

La **corrente diretta nominale** (*forward current*, I_f) è il valore di corrente per cui il LED è stato progettato per funzionare in modo ottimale e sicuro. Per i LED standard da 5 mm (quelli che useremo con Arduino), questo valore è tipicamente 20 mA.

Perché la corrente è così critica?

A differenza di una resistenza, il LED **non limita autonomamente la corrente che lo attraversa**. Se lo colleghiamo direttamente a una sorgente di tensione, la corrente tenderebbe ad aumentare senza controllo fino a distruggere il componente in pochi millisecondi. Per questo motivo è **sempre obbligatorio** inserire una resistenza di limitazione in serie al LED.

Corrente massima assoluta

La **corrente massima assoluta** è il valore oltre il quale il LED si danneggia in modo permanente e istantaneo. Per i LED standard è circa 30 mA. Non si deve mai avvicinarsi a questo valore: in pratica, per un uso sicuro con Arduino, è consigliabile lavorare con correnti tra 5 mA e 15 mA.

Nota pratica

I pin digitali di Arduino Uno possono erogare al massimo 40 mA per poco tempo, ma la scheda nel suo complesso non dovrebbe superare 200 mA totali. Per sicurezza, è buona norma progettare i circuiti per non superare i 10 mA per LED.

2.4 La resistenza di limitazione

Perché è necessaria

Abbiamo visto che il LED non può essere collegato direttamente alla tensione di alimentazione perché la corrente diventerebbe eccessiva. La soluzione è inserire in **serie** al LED una **resistenza di limitazione** R , il cui compito è quello di *assorbire* la differenza di tensione tra l'alimentazione e la V_f del LED, limitando di conseguenza la corrente.

Calcolo con la legge di Kirchhoff e la legge di Ohm

Consideriamo il circuito in serie composto da: sorgente di tensione V_{cc} , resistenza R e LED con tensione V_f .

Applicando la **legge di Kirchhoff alle tensioni** (la somma delle tensioni in un anello chiuso è zero), otteniamo:

$$V_{cc} = V_R + V_f \quad (2.1)$$

Un altro modo per vedere questa equazione è pensare che la tensione totale fornita da V_{cc} si divide tra la resistenza e il LED. La tensione che cade sulla resistenza, V_R , è quindi:

$$V_R = V_{cc} - V_f \quad (2.2)$$

In un circuito in serie la corrente è uguale in ogni punto, quindi la corrente che attraversa la resistenza è la stessa che attraversa il LED: $I_R = I_f$.

Applicando la **legge di Ohm** alla resistenza ($V = R \cdot I$):

$$R = \frac{V_R}{I_f} = \frac{V_{cc} - V_f}{I_f} \quad (2.3)$$

Esempio di calcolo

Vogliamo collegare un **LED rosso** al pin 13 di Arduino, che fornisce 5 V. Il LED rosso ha $V_f = 2.0$ V e vogliamo farlo lavorare a $I_f = 10$ mA per sicurezza.

$$R = \frac{V_{cc} - V_f}{I_f} = \frac{5 \text{ V} - 2.0 \text{ V}}{10 \text{ mA}} = \frac{3 \text{ V}}{0.010 \text{ A}} = 300 \Omega \quad (2.4)$$

Il valore esatto di 300Ω potrebbe non essere disponibile nella serie standard di resistenze commerciali. In questo caso scegliamo il valore **immediatamente superiore** disponibile, cioè 330Ω : una resistenza leggermente più alta riduce leggermente la corrente (e quindi la luminosità), ma protegge meglio il componente. Non si sceglie mai un valore inferiore, perché aumenterebbe la corrente oltre il valore di progetto.

Verifica del calcolo

Con $R = 330 \Omega$, la corrente effettiva sarà:

$$I_f = \frac{V_{cc} - V_f}{R} = \frac{5 \text{ V} - 2.0 \text{ V}}{330 \Omega} \approx 9.1 \text{ mA}$$

Valore ben al di sotto dei 20 mA nominali: il LED è al sicuro.

2.5 Schema di collegamento

Componenti necessari

Q.tà	Materiale
1	Generatore di tensione (es. Arduino Uno/Mega)
2	LED (colore a scelta)
2	Resistori
1	Breadboard
vari	Cavi e Jumpers

Schema elettrico

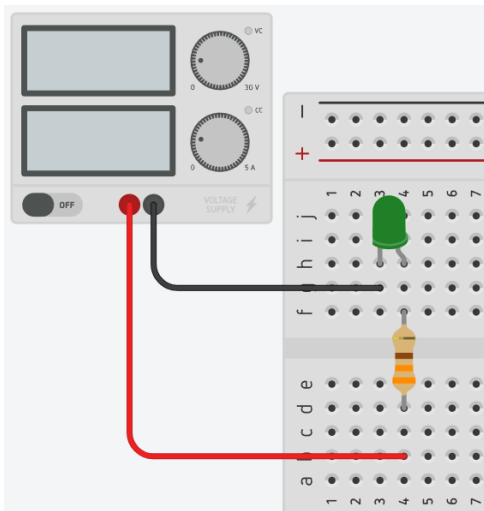


Figura 2.2: Circuito elettrico realizzato su Tinkercad.

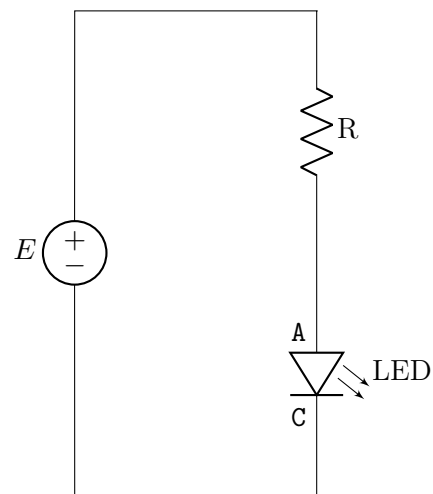


Figura 2.3: Schema elettrico.

Nelle figure (2.2) e (2.3) vediamo lo schema elettrico per accendere un LED con un generatore di tensione. Il terminale positivo del generatore è collegato all'anodo del LED attraverso una resistenza di limitazione, mentre il catodo del LED è collegato al terminale negativo del generatore. Dato che nell'esempio abbiamo scelto un LED verde, possiamo supporre che la tensione di polarizzazione diretta sia $V_f = 2.6 \text{ V}$ e che la corrente nominale sia $I_f = 20 \text{ mA}$. Se il generatore fornisce $V_{cc} = 5 \text{ V}$, la resistenza di limitazione da inserire in serie al LED sarà:

$$R = \frac{V_{cc} - V_f}{I_f} = \frac{5 \text{ V} - 2.6 \text{ V}}{20 \text{ mA}} = \frac{2.4 \text{ V}}{0.020 \text{ A}} = 120 \Omega \quad (2.5)$$

Una resistenza di 120Ω potrebbe non essere disponibile, quindi scegliamo il valore commerciale più vicino e superiore, cioè 150Ω , che garantisce una corrente di 16 mA , leggermente inferiore a quella nominale, ma comunque sufficiente per accendere il LED in modo visibile.

Se il circuito viene fatto utilizzando come generatore di tensione una scheda Arduino, è sempre meglio aumentare il valore di resistenza da 220Ω a 330Ω , in maniera da non stressare troppo i pin digitali di Arduino. Per verificare se il circuito è fatto correttamente, basta collegare il jumper che abbiamo precedentemente collegato alla boccia positiva dell'alimentatore al pin $+5 \text{ V}$ e collegare il jumper che abbiamo precedentemente collegato alla boccia negativa dell'alimentatore al pin GND di Arduino. Se tutto è stato fatto correttamente, il LED si accenderà. Se il led non si accende allora bisogna controllare se il circuito è stato montato correttamente prestando attenzione ai

vari collegamenti sulla breadboard e alla polarità del LED. Se continua a non accendersi se la resistenza che abbiamo scelto non sia troppo alta, oppure che il LED non sia danneggiato.

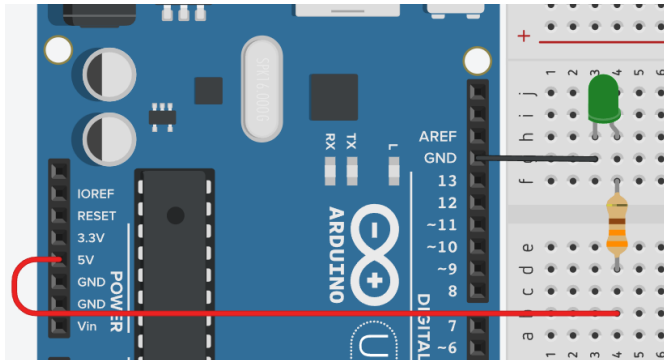


Figura 2.4: Circuito per verificare il funzionamento del circuito.

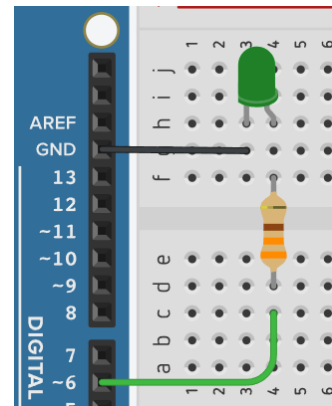


Figura 2.5: Circuito per accendere il LED con Arduino.

2.6 LED RGB

Il **LED RGB** è un componente che racchiude all'interno dello stesso componente tre LED: uno Rosso (*Red*), uno Verde (*Green*) e uno Blu (*Blue*). Controllando separatamente l'intensità di ciascuno di essi è possibile generare un'ampia gamma di colori, sfruttando il principio della sintesi additiva dei colori.

Struttura interna e tipologie

Un LED RGB ha quattro terminali: tre anodi (uno per ciascun colore) e un catodo comune, oppure tre catodi e un anodo comune. Le due configurazioni possibili sono:

- **Anodo comune** (Common Anode): tutti e tre gli anodi sono collegati insieme al terminale positivo. Per accendere un colore, si porta a massa (GND) il catodo corrispondente.
- **Catodo comune** (Common Cathode): tutti e tre i catodi sono collegati insieme al terminale negativo. Per accendere un colore, si applica una tensione positiva all'anodo corrispondente.

Quale usare con Arduino?

La maggior parte dei kit didattici utilizza LED RGB a **catodo comune**. In questo caso il pin comune va collegato a GND, e i tre pin dei colori vanno collegati ad Arduino attraverso tre resistenze di limitazione (una per canale). Per un LED RGB ad anodo comune, il pin comune va collegato a 5V e i pin dei colori vanno portati a GND per accendere il LED (logica invertita).

Generazione dei colori

I tre LED interni possono essere accesi singolarmente o in combinazione. La tabella seguente mostra i colori ottenuti accendendo a piena intensità diverse combinazioni (per catodo comune):

Colore risultante	Rosso (R)	Verde (G)	Blu (B)
Rosso	HIGH	LOW	LOW
Verde	LOW	HIGH	LOW
Blu	LOW	LOW	HIGH
Ciano	LOW	HIGH	HIGH
Magenta	HIGH	LOW	HIGH
Giallo	HIGH	HIGH	LOW
Bianco	HIGH	HIGH	HIGH
Spento	LOW	LOW	LOW

Variando l'intensità di ciascun canale tramite PWM si ottengono infinite sfumature. Ad esempio, un rosso intenso si ottiene con $R=255$, $G=0$, $B=0$; un arancione si può ottenere con $R=255$, $G=80$, $B=0$. Potendo usare 256 livelli di intensità per ciascun colore del LED RGB, si possono generare $256^3 = 16.777.216$ combinazioni diverse.

Schema di collegamento

Collegiamo un LED RGB a catodo comune ai pin digitali di Arduino. Utilizziamo tre resistenze di limitazione, una per ogni colore. Il calcolo della resistenza è identico a quello visto per il LED singolo, usando la V_f di ogni colore (tipicamente: rosso 2.0 V, verde 2.2 V, blu 3.2 V). Per semplicità, con alimentazione a 5 V e una corrente di circa 10 mA, si possono usare resistenze da $330\ \Omega$ per tutti e tre i canali.

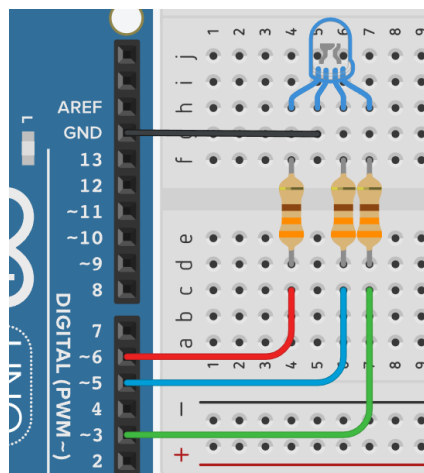


Figura 2.6: Schema di collegamento di un LED RGB a catodo comune con Arduino.

Ricordiamo che per poter controllare la luminosità di ciascun colore è necessario utilizzare i pin digitali con funzionalità PWM e la funzione `analogWrite()` per variare il duty cycle del segnale.

2.7 Programmazione con Arduino

Configurare un pin come uscita: `pinMode()`

Prima di poter controllare un LED (o qualsiasi altro componente), dobbiamo dire ad Arduino se quel pin digitale deve comportarsi da **uscita** (Arduino manda tensione al circuito) o da **ingresso** (Arduino legge la tensione dal circuito). Questo si fa con la funzione che dichiara la modalità di utilizzo del pin, cioè la funzione `pinMode()`:

```
pinMode(numeroPIN, modalità);
```

- **numeroPIN**: il numero del pin da configurare (es. 6, come mostrato in figura 2.5)
- **modalità**: OUTPUT per uscita, INPUT per ingresso. Per accendere un LED, bisogna inviare un segnale dall'Arduino all' "esterno", quindi configureremo il pin come pin di OUTPUT.

Quindi per impostare il pin 6 come pin usato per accendere il led in figura 2.5 bisogna scrivere:

```
pinMode(6, OUTPUT);
```

Accendere e spegnere il LED: digitalWrite()

Per portare un pin digitale a 5 V (LED acceso) o a 0 V (LED spento) si usa digitalWrite():

```
digitalWrite(numeroPIN, valore);
```

- **HIGH**: porta il pin a 5 V → LED **acceso**
- **LOW**: porta il pin a 0 V → LED **spento**

Quindi per accendere il LED collegato al pin 6 bisogna scrivere:

```
digitalWrite(6, HIGH);
```

Mentre per spegnere il led bisognerà scrivere:

```
digitalWrite(6, LOW);
```

2.7.1 Lampeggio del LED e funzione delay/(ms);

Proviamo subito a far lampeggiare un LED, accendendolo e spegnendolo all'interno della funzione void loop(). Abbiamo anche bisogno di dichiarare che il pin dove verrà collegato il LED è un pin di OUTPUT, quindi scriveremo pinMode() all'interno della funzione void setup(). Quindi il codice completo per far lampeggiare un LED collegato al pin 6 dovrebbe essere il seguente:

Esempio 1 - Blink NON funzionante

```
1 void setup() {  
2     pinMode(6, OUTPUT);  
3 }  
4  
5 void loop() {  
6     digitalWrite(6, HIGH); // Accende LED (porta il pin a 5V)  
7     digitalWrite(6, LOW);  // Spegne LED (porta il pin a 0V)  
8 }
```

Se proviamo a caricare questo *sketch* su Arduino, vedremo che il funzionamento atteso non rispecchia quello che stiamo notando sul nostro LED: il LED si accende e si spegne così rapidamente che sembra essere sempre acceso. Ricordiamo che la funzione void loop() viene eseguita in modo continuo e ciclico, quindi il LED viene acceso e spento migliaia di volte al secondo. Questa velocità di esecuzione è troppo elevata per essere percepita dall'occhio umano, che appunto vede il LED sempre acceso.

Per risolvere questo problema, dobbiamo inserire una pausa tra l'accensione e lo spegnimento del LED. La funzione `delay()` ci permette di bloccare l'esecuzione del programma per un certo numero di **millisecondi**. Se vogliamo che il LED rimanga acceso per 1 secondo e poi spento per 1 secondo, dobbiamo inserire un `delay(1000);` dopo ogni comando di accensione o spegnimento.

Inoltre, aggiungiamo in questo codice una riga per creare una variabile che contiene il numero del pin a cui è collegato il LED, in modo da rendere il codice più leggibile e facilmente modificabile in futuro.

Esempio 2 - Blink

```
1  const int ledPin = 6;  // Pin a cui è collegato il LED
2
3  void setup() {
4      pinMode(ledPin, OUTPUT);
5  }
6
7  void loop() {
8      digitalWrite(ledPin, HIGH);
9      delay(1000);        // Aspetta 1000 millisecondi (1 secondo)
10     digitalWrite(ledPin, LOW);
11     delay(1000);        // Aspetta 1000 millisecondi (1 secondo)
12 }
```

Analisi del codice riga per riga:

- **Riga 1** - `const int ledPin = 6;`: dichiara una variabile costante di tipo intero chiamata `ledPin` e le assegna il valore 6, che rappresenta il numero del pin digitale a cui è collegato il LED.
 - `int` indica che la variabile è di tipo intero
 - `const` indica che il valore di questa variabile non può essere modificato durante l'esecuzione del programma, se per errore si tenta di assegnare un nuovo valore a `ledPin`, il compilatore genererà un errore, proteggendo così la costante da modifiche accidentali.
 - L'uso di una variabile con nome significativo come `ledPin` rende il codice più leggibile e facile da capire, rispetto a usare direttamente il numero 6 in tutto il programma. Si può facilmente modificare il numero del pin in un solo punto (la dichiarazione della variabile) se si decide di collegare il LED a un pin diverso in futuro, senza dover cercare e sostituire ogni occorrenza del numero 6 nel codice.

, mentre

- **Riga 4** - `pinMode(ledPin, OUTPUT);`: eseguita una sola volta all'avvio, configura il pin `ledPin` come uscita digitale.
- **Riga 8** - `digitalWrite(ledPin, HIGH);`: porta il pin 13 a 5V. La corrente scorre attraverso la resistenza e il LED, che si accende.
- **Riga 9 e Riga 11** - `delay(1000);`: il programma si blocca per 1000 ms. Durante questo tempo il LED rimane nello stato in cui si trova.
- **Riga 10** - `digitalWrite(ledPin, LOW);`: porta il pin `ledPin` a 0V. Nessuna differenza di tensione, nessuna corrente, LED spento.

2.7.2 Variare la luminosità: PWM e analogWrite()

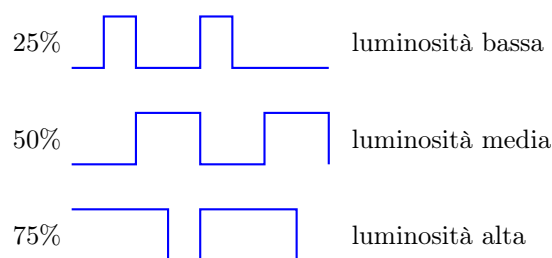
I pin digitali di Arduino possono essere solo in due stati: 5 V (HIGH) oppure 0 V (LOW) . Non è possibile impostare direttamente una tensione intermedia di, ad esempio, 2.5 V. Come facciamo allora a variare la luminosità del LED?

La soluzione si chiama **PWM** (*Pulse Width Modulation*, Modulazione di Larghezza di Impulso). L'idea è la seguente: invece di tenere il pin fisso a un valore di tensione intermedio, il pin viene commutato rapidamente tra HIGH e LOW a una frequenza molto alta. L'occhio umano, non riuscendo a percepire variazioni così rapide, vede una luminosità *media* che dà l'impressione di essere proporzionale al tempo in cui il pin è rimasto HIGH.

Questo rapporto tra il tempo t_H in cui il segnale è HIGH e il periodo totale si chiama **duty cycle**:

$$\text{Duty Cycle} = \frac{t_H}{T} \times 100\% \quad (2.6)$$

Un duty cycle del 50% significa che il pin è (HIGH) per metà del tempo: il LED appare a metà della sua luminosità massima. Un duty cycle del 100% equivale a `digitalWrite(pin, HIGH)`: LED alla massima luminosità.



La funzione `analogWrite(pin, valore);` imposta il duty cycle del pin PWM specificato. Il parametro `valore` va da **0** (duty cycle 0%, LED completamente spento) a **255** (duty cycle 100%, LED alla massima luminosità). Se quindi vogliamo accendere il LED al 32% dobbiamo ricorrere ad una semplice proporzione:

$$\text{valore} : 255 = 32 : 100 \quad \Rightarrow \quad \text{valore} = \frac{32}{100} \times 255 \approx 82 \quad (2.7)$$

Il numero massimo di 255 è dovuto al fatto che il duty cycle viene rappresentato internamente con un numero a 8 bit (1 byte), che può assumere 256 valori (da 0 a 255).

Quali pin supportano il PWM?

Su Arduino Uno, **non tutti i pin** supportano il PWM. I pin abilitati al PWM sono contrassegnati dal simbolo ~ sulla scheda e sono: **3, 5, 6, 9, 10, 11**. Se si tenta di usare `analogWrite()` su un pin non PWM, il comportamento è imprevedibile.

analogWrite non vuol dire pin analogico!

`analogWrite()`; è una funzione che si applica solo ai pin digitali abilitati al PWM, non ai pin analogici. I pin analogici di Arduino (quelli identificati con una A e un numero) sono progettati solamente per leggere segnali analogici (con `analogRead()`) e non possono essere usati per generare un segnale PWM.

Quindi possiamo ora scrivere un programma che accende un LED al 20% di luminosità per 1 secondo, poi lo spegne, poi lo riaccende ma al 80% di luminosità per 1 secondo, poi lo rispegne e così via:

Esempio 3 - Variazione luminosità

```
1  const int pinLed = 6;
2
3  void setup() {
4      pinMode(pinLed, OUTPUT);
5  }
6
7  void loop() {
8      analogWrite(pinLed, 51); // Accende il LED con duty cycle 20 %
9      delay(1000);
10     analogWrite(pinLed, 0); // Spegne il LED con duty cycle 0 %
11     delay(1000);
12     analogWrite(pinLed, 204); // Accende il LED con duty cycle 80 %
13     delay(1000);
14     analogWrite(pinLed, 0); // Spegne il LED con duty cycle 0 %
15     delay(1000);
16 }
```

2.7.3 LED RGB

Grazie al PWM possiamo controllare non solo la luminosità di un singolo LED, ma anche il colore di un LED RGB, miscelando in proporzioni variabili l'intensità dei tre canali Rosso, Verde e Blu.

Accendere un colore primario

Per ottenere rosso, verde o blu puri, si accende a pieno il canale corrispondente (valore 255) e si spengono gli altri (valore 0). Ad esempio, per il rosso:

Colore rosso

```
1  const int PIN_G = 3; // Pin Verde
2  const int PIN_B = 5; // Pin Blu
3  const int PIN_R = 6; // Pin Rosso
4
5  void setup() {
6      pinMode(PIN_G, OUTPUT);
7      pinMode(PIN_B, OUTPUT);
8      pinMode(PIN_R, OUTPUT);
9  }
10 void loop() {
11     analogWrite(PIN_R, 255);
12     analogWrite(PIN_G, 0);
13     analogWrite(PIN_B, 0);
14     delay(2000);
15 }
```


Capitolo 3

Pulsante

3.1 Introduzione

Il **pulsante** (o *push button*) è uno dei componenti di input più semplici e comuni nel mondo dell'elettronica. Viene utilizzato per inviare un comando dall'utente al sistema: può accendere o spegnere una luce, avviare un processo, selezionare un'opzione e molto altro. Lo troviamo quotidianamente su telecomandi, tastiere, suonerie di campanelli, pulsanti di accensione e innumerevoli altre applicazioni.

In questo capitolo impareremo come collegare correttamente un pulsante ad Arduino e come scrivere il codice per leggere il suo stato (premuto o rilasciato), gestendo il fenomeno del *rimbalzo* meccanico dei contatti (*bouncing*).



Figura 3.1: Classico pulsante a 4 pin.

3.2 Struttura e funzionamento

Come è fatto un pulsante

Un pulsante è un interruttore momentaneo: quando viene premuto, chiude un circuito elettrico; quando viene rilasciato, lo riapre. La struttura interna più semplice è composta da quattro terminali (pin). In realtà, i terminali sono collegati a coppie: due terminali sono sempre connessi tra loro internamente, e gli altri due sono sempre connessi tra loro. Quando si preme il pulsante, le due coppie si collegano, chiudendo il circuito.

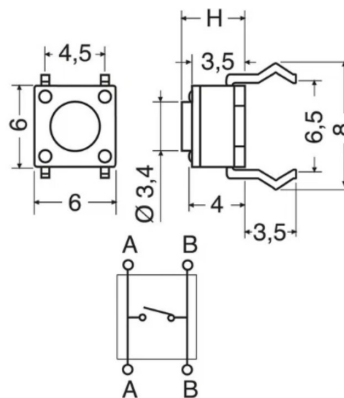


Figura 3.2: Struttura interna di un pulsante a 4 pin.

I pulsanti utilizzati nei kit didattici hanno spesso quattro piedini disposti su due lati. I due piedini sullo stesso lato sono internamente collegati. Quindi, per collegare un pulsante ad Arduino, è sufficiente utilizzare un piedino da ciascuna coppia, ad esempio uno del lato A e uno del lato B. In questo modo, quando si preme il pulsante, si chiude il circuito tra i due piedini utilizzati.

Stati del pulsante

Dal punto di vista elettrico, un pulsante può trovarsi in due stati:

- **Aperto (rilasciato):** i contatti interni non si toccano, il circuito è aperto e non scorre corrente.
- **Chiuso (premuto):** i contatti interni si toccano, il circuito è chiuso e la corrente può scorrere.

Esercizi proposti

1. **Continuità:** Utilizzando un multimetro in modalità di continuità, misurare la resistenza tra i pin del pulsante quando è premuto e quando è rilasciato.
2. **Circuito con LED:** Collegare un LED e una resistenza in serie al pulsante, alimentandolo con un generatore impostato a 9 V. Verificare che il LED si accenda solo quando il pulsante è premuto.

3.3 Arduino e resistenze di pull-up e pull-down

Perché sono necessarie

Un pin di ingresso digitale di Arduino deve sempre vedere una tensione ben definita: **HIGH** (vicino a 5 V) o **LOW** (vicino a 0 V). Se il pin non è collegato a nulla (floating), può captare disturbi elettromagnetici e leggere valori casuali. La resistenza di *pull-up* o *pull-down* fissa il pin a uno stato logico noto quando il pulsante non è premuto.

Configurazione pull-down

Nel circuito **pull-down**, il pin di ingresso è collegato a massa (GND) tramite una resistenza di alto valore (tipicamente 10 k Ω). Il pulsante collega il pin a 5 V quando viene premuto.

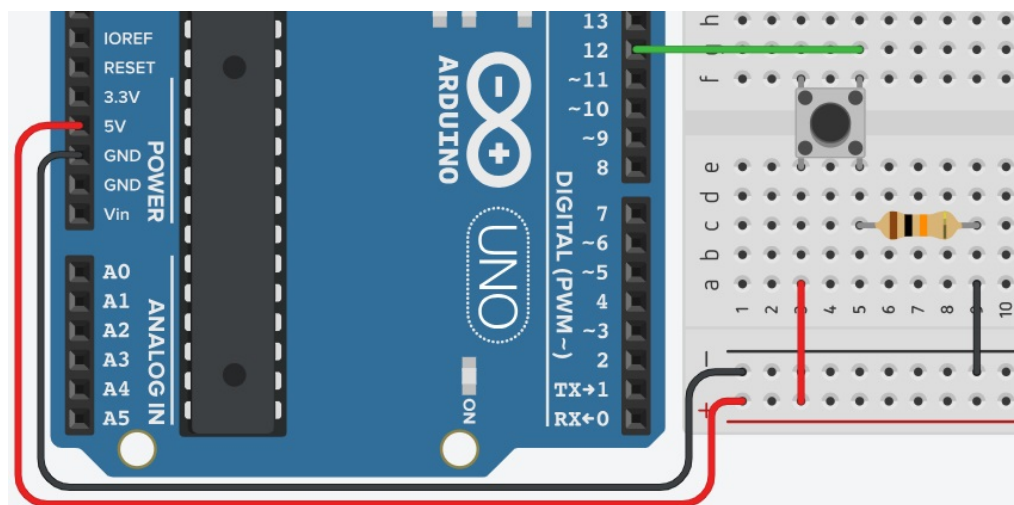


Figura 3.3: Collegamento pull-down: pin LOW a riposo, HIGH quando premuto.

- **Pulsante non premuto:** il pin è collegato a GND tramite la resistenza → legge LOW.
- **Pulsante premuto:** il pin è collegato direttamente a 5 V (la resistenza è in parallelo ma la corrente preferisce il percorso a bassa resistenza del pulsante) → legge HIGH.

Configurazione pull-up

Nel circuito **pull-up**, il pin di ingresso è collegato a 5 V tramite una resistenza. Il pulsante collega il pin a GND quando viene premuto.

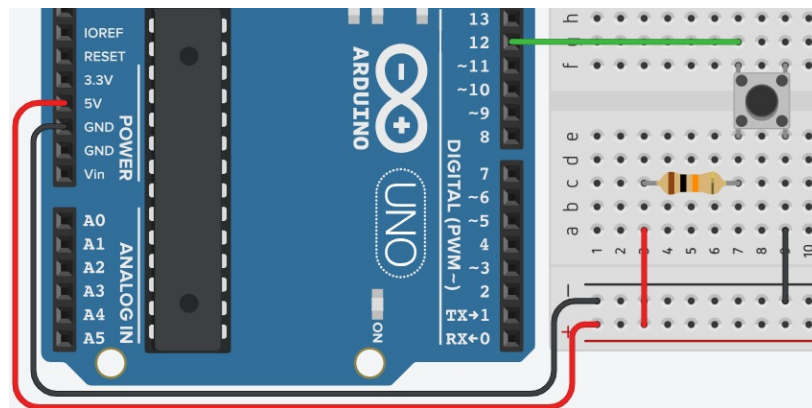


Figura 3.4: Collegamento pull-up: pin HIGH a riposo, LOW quando premuto.

- **Pulsante non premuto:** il pin è collegato a 5 V tramite la resistenza → legge HIGH.
- **Pulsante premuto:** il pin è collegato direttamente a GND → legge LOW.

Attenzione: importanza del resistore

Non collegare un pulsante direttamente tra 5 V e GND **senza una resistenza**, altrimenti si crea un cortocircuito quando il pulsante è premuto, che può danneggiare Arduino.

3.3.1 Pull-Up interno ad arduino

Sui pin digitali di Arduino è possibile attivare una resistenza di pull-up interna, evitando di doverla collegare esternamente. Per farlo, basta configurare il pin come `INPUT_PULLUP` nel codice. In questo modo, il pin sarà collegato internamente a 5 V tramite una resistenza di pull-up, e il pulsante dovrà collegare il pin a GND per essere letto come LOW quando premuto.

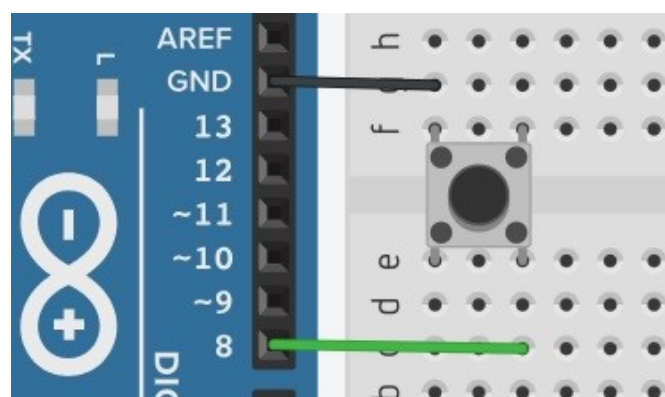


Figura 3.5: Collegamento pull-up interno ad Arduino

3.4 Programmazione con Arduino

Leggere lo stato del pulsante: `digitalRead()`

Per leggere il valore logico presente su un pin digitale si usa la funzione `digitalRead()`, che restituisce `HIGH` o `LOW`.

```
int valore = digitalRead(numeroPIN);
```

Il valore restituito è `HIGH` se sul pin è presente una tensione di circa 5 V, altrimenti se la tensione è vicina a 0 V allora verrà restituito `LOW`.

Leggere e stampare sul seriale

Collegiamo il pulsante al pin 2 con pull-up interno e inviamo lo stato al monitor seriale.

Esempio 1 - Lettura semplice

```
1 // Pin a cui è collegato il pulsante
2 const int buttonPin = 2;
3 // Variabile per memorizzare lo stato
4 bool buttonState = LOW;
5
6 void setup() {
7     Serial.begin(9600);
8     pinMode(buttonPin, INPUT_PULLUP); // Pin con pull-up interno
9 }
10
11 void loop() {
12     // Legge stato pulsante
13     buttonState = digitalRead(buttonPin);
14
15     // se lo stato è uguale a LOW, vuol dire che è stato premuto
16     if (buttonState == LOW) {
17         Serial.println("Pulsante premuto");
18     } else {
19         Serial.println("Pulsante rilasciato");
20     }
21
22     // Piccola pausa per evitare troppi messaggi
23     delay(50);
24 }
```

3.5 Il problema del rimbalzo - Bouncing

Creiamo un codice per contare quante volte il pulsante viene premuto. L'idea è semplice: ogni volta che rileviamo una transizione da `HIGH` a `LOW` (pulsante premuto), incrementa un contatore.

Esempio 2 - Contatore pressioni

```
1 const int buttonPin = 2; // Pin a cui è collegato il pulsante
2 bool stato_attuale = HIGH; // Memorizza lo stato attuale
3 bool stato_precedente = HIGH; // Memorizza lo stato precedente
4 int contatore = 0; // Contatore di pressioni
5
```



```
6 void setup() {  
7     Serial.begin(9600);  
8     pinMode(buttonPin, INPUT_PULLUP); // Pin con pull-up interno  
9 }  
10  
11 void loop() {  
12     // Legge stato pulsante  
13     stato_attuale = digitalRead(buttonPin);  
14  
15     // se lo stato è cambiato, vuol dire che è stato premuto  
16     if (stato_attuale == LOW && stato_precedente == HIGH) {  
17         contatore++;  
18         Serial.print("Pulsante premuto ");  
19         Serial.print(contatore);  
20         Serial.println(" volte");  
21     }  
22  
23     // Aggiorna stato precedente  
24     stato_precedente = stato_attuale;  
25 }
```

Attenzione: il risultato non è quello atteso!

Proviamo adesso a premere il pulsante esattamente 30 volte. Se tutto funziona correttamente, l'ultima stampa dovrebbe essere "Pulsante premuto 30 volte" sul monitor seriale. Tuttavia potremmo vedere un numero più alto di 30 anche se abbiamo premuto il pulsante solo 30 volte.

Quando il pulsante viene premuto o rilasciato, i contatti meccanici non si chiudono o aprono in modo pulito: rimbalzano per alcuni millisecondi (tipicamente 5–20 ms), generando una rapida sequenza di transizioni HIGH-LOW-HIGH. Arduino legge questi rimbalzi come molte pressioni anziché una sola.

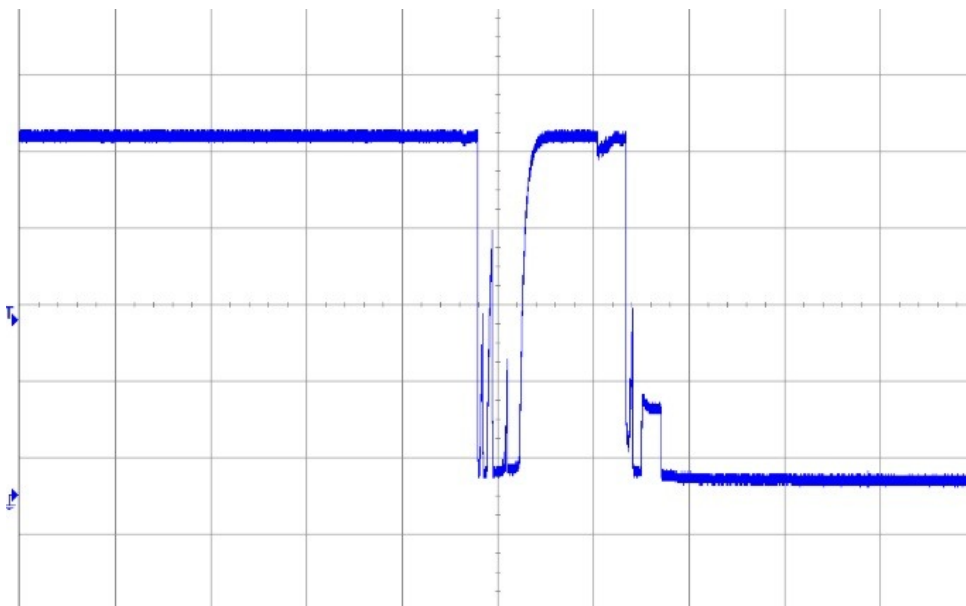


Figura 3.6: Rappresentazione del rimbalzo meccanico dei contatti di un pulsante.

Soluzione al problema del bouncing: Bounce2

Esiste una libreria chiamata **Bounce2** che implementa un algoritmo di debouncing efficace. La libreria filtra i rimbalzi e fornisce metodi per rilevare quando il pulsante è stato effettivamente premuto o rilasciato.

Per installare la libreria Bounce2, apri l'IDE di Arduino, nel menù a sinistra c'è un'icona con dei libri (Gestore librerie). Cliccaci sopra, poi nella barra di ricerca scrivi "Bounce2" e installa la libreria sviluppata da Thomas Ouellet Fredericks.

Questa libreria utilizza un approccio basato su timer per ignorare i rimbalzi. Quando viene rilevata una transizione, la libreria aspetta un intervallo di tempo predefinito (ad esempio 50 ms) prima di considerare la transizione come valida. Se durante questo intervallo si verificano altre transizioni, vengono ignorate.

Vediamo un esempio di utilizzo della libreria Bounce2 per contare le pressioni di un pulsante senza essere influenzati dai rimbalzi.

Esempio - Utilizzo libreria Bounce2 con pulsante

```
1 // Libreria per eliminare i rimbalzi del pulsante
2 #include <Bounce2.h>
3
4 const int buttonPin = 2;
5 int contatore = 0;
6
7 // Oggetto che gestisce il pulsante con filtro anti-rimbalzo
8 Bounce debouncer = Bounce();
9
10 void setup() {
11     pinMode(buttonPin, INPUT_PULLUP);
12     Serial.begin(9600);
13
14     // Colleghiamo la libreria al pin del pulsante
15     debouncer.attach(buttonPin);
16
17     // Impostiamo 50 ms come tempo di stabilizzazione del segnale
18     debouncer.interval(50);
19
20 }
21
22 void loop() {
23     // Aggiorna continuamente lo stato del pulsante
24     debouncer.update();
25
26     // Se pulsante è stato premuto, stampa il contatore aggiornato
27     if (debouncer.fell()) {
28         contatore++;
29         Serial.println(contatore);
30     }
31 }
```

La libreria utilizzata risolve questo problema controllando il segnale nel tempo: quando rileva un cambiamento, aspetta che il segnale si stabilizzi per un breve intervallo (in questo caso 50 millisecondi). Solo dopo questo controllo considera la pressione valida. Nel programma, ogni volta che viene rilevata una pressione reale del pulsante, viene aumentata di uno una variabile chiamata contatore. In questo modo si può sapere quante volte il pulsante è stato premuto in modo affidabile.

Funzioni importanti della libreria Bounce2 DA MODIFICARE

La libreria **Bounce2** permette di gestire in modo affidabile i pulsanti eliminando i problemi causati dai rimbalzi elettrici. Tra i metodi più importanti c'è `attach()`, che serve a collegare la libreria a un determinato pin e a impostarne la modalità di funzionamento (come ingresso normale o con resistenza di pull-up). Un altro metodo fondamentale è `update()`, che deve essere chiamato continuamente nel ciclo principale del programma per aggiornare lo stato del pulsante e applicare il filtro anti-rimbalzo.

Per definire il tempo di stabilizzazione del segnale si utilizza `interval()`, che imposta per quanti millisecondi il segnale deve restare stabile prima di essere considerato valido. Per sapere se il pulsante è premuto in un determinato momento si può usare `isPressed()`, mentre `pressed()` e `released()` servono per rilevare rispettivamente l'istante in cui il pulsante viene premuto o rilasciato.

Molto utili sono anche `fell()` e `rose()`, che permettono di rilevare le transizioni del segnale: `fell()` indica il passaggio da alto a basso (tipico di una pressione con pull-up), mentre `rose()` indica il passaggio da basso ad alto. Infine, metodi come `changed()` permettono di sapere se lo stato del pulsante è cambiato dall'ultimo aggiornamento, mentre `read()` restituisce semplicemente lo stato attuale del pin.

Rilevare la durata della pressione

Possiamo misurare per quanto tempo il pulsante rimane premuto usando `millis()`.

Esempio 4 - Durata pressione

```
1  const int buttonPin = 2;
2  unsigned long pressStart = 0;
3  bool wasPressed = false;
4
5  void setup() {
6      Serial.begin(9600);
7      pinMode(buttonPin, INPUT_PULLUP);
8  }
9
10 void loop() {
11     bool isPressed = (digitalRead(buttonPin) == LOW);
12
13     if (isPressed && !wasPressed) {
14         pressStart = millis();           // inizio pressione
15         wasPressed = true;
16     }
17     else if (!isPressed && wasPressed) {
18         unsigned long duration = millis() - pressStart;
19         Serial.print("Pulsante premuto per ");
20         Serial.print(duration);
21         Serial.println(" ms");
22         wasPressed = false;
23     }
24     delay(10);
25 }
```

Attenzione: `delay()` blocca il programma

L'uso di `delay()` per il debouncing blocca l'esecuzione del programma, impedendo di leggere altri sensori o eseguire azioni contemporanee. Per applicazioni più avanzate, utilizzare tecniche non bloccanti basate su `millis()`.